

Algoritmo Genético Distribuído para o Problema do Caixeiro Viajante

Alberto Magno, Josué Pereira, Leonardo Saraiva, Leonardo Pizani, Pedro Henrique Caillaux

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)

Resumo

Este trabalho apresenta o projeto de um sistema distribuído para resolução do Problema do Caixeiro Viajante (PCV) utilizando um Algoritmo Genético baseado no Modelo de Ilhas. A solução distribui 1000 rotas candidatas entre cinco nós computacionais, cada um responsável por evoluir localmente sua subpopulação e enviar os melhores indivíduos a um Coordenador Central. O coordenador opera de forma assíncrona com threshold de 80%, seleciona as melhores rotas globais e as redistribui com filtro de origem para preservar a diversidade genética. A comunicação é realizada via gRPC com Protocol Buffers, a ordenação causal de eventos é garantida por Relógios Lógicos de Lamport e a tolerância a falhas é assegurada pelo Algoritmo do Valentão para eleição dinâmica de líder.

Palavras-chave: Algoritmo Genético Distribuído, Caixeiro Viajante, gRPC, Lamport, Bully, Modelo de Ilhas.

1. Explicação do Problema Escolhido

O problema central abordado neste trabalho é a resolução do Problema do Caixeiro Viajante (PCV) por meio de um Algoritmo Genético Distribuído. O PCV consiste em determinar a rota de menor distância total que permita a um viajante partir de uma cidade de origem, percorrer um conjunto de cidades exatamente uma vez e retornar ao ponto de partida, formando um ciclo fechado.

O PCV é classificado como NP-difícil: para N cidades existem $(N-1)!/2$ rotas distintas possíveis, tornando a busca por solução exata inviável para instâncias de maior escala. Para contornar essa complexidade, utiliza-se uma heurística baseada em Algoritmo Genético (AG), com operadores de seleção, cruzamento e mutação para evoluir populações de rotas candidatas em direção a soluções de alta qualidade.

Cada indivíduo representa uma rota completa (permutação das cidades). A função de fitness avalia a qualidade pelo inverso da distância total percorrida, incluindo o retorno à cidade de origem:

```
fitness(rota) = 1 /  
distancia_total(rota)
```

A implementação adota o Modelo de Ilhas: a população é distribuída entre nós computacionais independentes que trocam periodicamente os melhores indivíduos via migração, evitando convergência prematura e potencializando a exploração paralela do espaço de soluções.

2. Arquitetura do Sistema

O sistema é composto por cinco nós computacionais (M1 a M5), sendo M2 designado como Coordenador Central. Toda a comunicação usa gRPC com Protocol Buffers. Uma população de 1000 rotas candidatas é distribuída igualmente: cada nó recebe e processa 200 indivíduos. As coordenadas das cidades são fixas e conhecidas por todos os nós desde o início da execução.

2.1. Papel de cada entidade

Nós trabalhadores (M1, M3, M4, M5) evoluem sua subpopulação local por N gerações sem comunicação externa. Ao término, selecionam os 30 mais aptos e os enviam ao coordenador via gRPC. Ao receber migrantes externos, substituem seus 30 piores indivíduos e reiniciam o ciclo.

Coordenador (M2) acumula função dupla: evolui sua subpopulação local e gerencia a migração global. Opera de forma assíncrona, processando ao atingir o threshold de 80% (mínimo 4 de 5 nós). Aplica filtro de origem na redistribuição: cada nó recebe apenas indivíduos gerados por outros nós, preservando a diversidade genética entre as ilhas.

2.2. Operadores genéticos e inicialização

A população inicial de cada nó é gerada por permutações aleatórias das cidades. O cruzamento adotado é o Order Crossover (OX), que preserva a ordem relativa das cidades e evita duplicatas. A mutação aplica troca de posição (swap): duas cidades aleatórias têm suas posições invertidas. A seleção usa torneio para escolher pais e elitismo puro para preservar os melhores indivíduos entre gerações consecutivas.

2.3. Critério de parada

O sistema encerra quando o melhor fitness global não apresentar melhora superior a 0,001 por 10 ciclos consecutivos. Um número máximo absoluto de ciclos é configurado como salvaguarda, garantindo terminação dentro de tempo razoável independentemente do comportamento da população.

2.4. Mensagens trocadas

EnviarMelhores — enviada por cada nó ao Coordenador ao final de cada ciclo:

Campo	Tipo	Descrição
indivíduos	repeated Individuo	30 melhores indivíduos do ciclo
lamport_ts	int64	Timestamp lógico de Lamport
origem	string	Identificador do nó remetente

ciclo	int32	Número do ciclo atual
-------	-------	-----------------------

Cada **Individuo** contém:

Campo	Tipo	Descrição
genes	repeated string	Sequência de cidades da rota
aptidao	float	1 / distancia_total(rota)
geracao	int32	Geração em que foi produzido

ReceberMigrantes — enviada pelo Coordenador com os indivíduos externos selecionados do pool global (filtro de origem aplicado) e o timestamp lógico do envio.

MensagemEleicao / MensagemLider — trocadas entre nós durante a eleição, carregando os identificadores dos participantes e, na segunda, o novo líder eleito.

2.5. Fluxo de comunicação por ciclo

```
M1..M5 ==> EnviarMelhores ==> Coord. (M2)
Coord. aguarda threshold >= 80% (min. 4 de 5 nos)
Coord. consolida pool e aplica filtro de origem
Coord. ==> ReceberMigrantes ==> cada no
(cada no recebe apenas individuos externos)
Cada no substitui 30 piores e reinicia evolucao
```

2.6. Detecção de falha e eleição de líder

Cada nó mantém uma thread de monitoramento que verifica periodicamente se o coordenador responde ao método ObterStatus. Timeout sem resposta dispara o Algoritmo do Valentão: o nó envia MensagemEleicao aos nós de ID maior; se nenhum responder, proclama-se líder e notifica todos com MensagemLider. O novo coordenador assume imediatamente sem interrupção do processo evolutivo.

3. Detalhamento da Implementação

3.1. Comunicação com gRPC e Protocol Buffers

A comunicação entre as ilhas e o coordenador foi construída com gRPC, cujo contrato é definido em arquivo .proto. O serviço expõe cinco métodos RPC:

Método RPC	Direção	Descrição
EnviarMelhores	Nó => Coord.	30 melhores com metadados de controle
ReceberMigrantes	Coord. => Nó	Indivíduos externos do pool global
ReceberEleicao	Nó => Nó	Inicia processo de eleição

ReceberLider	Nó => Nó	Anuncia novo coordenador eleito
ObterStatus	Nó => Coord.	Verificação de disponibilidade

O uso de Protocol Buffers garante serialização binária eficiente e tipagem forte, fundamental para transmitir estruturas complexas como sequências de genes. O modelo assíncrono permite que nós realizem envios independentes ao atingir critérios de evolução, sem bloquear uns aos outros.

3.2. Relógios Lógicos de Lamport

A necessidade do Lamport surge da arquitetura assíncrona: como o coordenador processa com threshold de 80%, nós atrasados podem enviar indivíduos de gerações anteriores após o pool já ter sido consolidado com dados mais recentes.

A regra de atualização segue o protocolo clássico: cada nó mantém um contador local inicializado em zero; a cada evento relevante o contador é incrementado em uma unidade; ao receber mensagem com timestamp T, o receptor atualiza para max(local, T) + 1. Isso garante que eventos causalmente relacionados possuam sempre timestamps crescentes, independentemente do relógio físico de cada máquina.

Na prática, o coordenador compara o lamport_ts de cada mensagem com o timestamp já registrado para aquela origem no pool. Dados com timestamp inferior são descartados, evitando que populações obsoletas contaminem a seleção global e prejudiquem a qualidade da solução.

3.3. Tolerância a Falhas — Algoritmo do Valentão (Bully)

O Algoritmo do Valentão garante a eleição dinâmica de um novo coordenador sem intervenção manual. O fluxo ocorre em cinco etapas:

- (1) Não detecta falha por timeout na chamada ObterStatus;
- (2) Envia MensagemEleicao a todos os nós de ID maior;
- (3) Se algum de ID maior responder, ele assume a eleição;
- (4) Se nenhum responder, o nó se proclama líder;
- (5) Envia MensagemLider a todos os nós ativos.

O principal desafio é distinguir falhas reais de atraso de comunicação. O timeout é um parâmetro configurável, calibrado conforme as características da rede local. Valor muito baixo provoca eleições espúrias; valor muito alto mantém o sistema parado desnecessariamente.

4. Desafios e Problemas Encontrados

A implementação apresentou desafios relacionados principalmente à adaptação do Algoritmo Genético para ambiente distribuído. Como cada nó mantém

apenas parte da população total, foi necessário definir uma forma eficiente de trocar indivíduos sem comprometer a diversidade genética nem gerar dependência excessiva do coordenador.

Controle de concorrência. Como o gRPC processa requisições em threads paralelas, múltiplos nós podem tentar atualizar o pool global simultaneamente, exigindo mecanismos de exclusão mútua para garantir a integridade dos dados sem comprometer o desempenho. Ademais, o uso de locks em momentos de escrita sobre partituras de memória compartilhada, protege o sistema contra sobrescritas e obtenção de valores desatualizados.

Threshold assíncrono e dados obsoletos. O threshold de 80% evita bloqueio por nós lentos, mas nós atrasados podem enviar dados de ciclos anteriores desatualizados. O Lamport foi adotado para identificar e descartar esses dados de forma sistemática.

Convergência prematura. Migração muito frequente ou com muitos indivíduos homogeneiza rapidamente as ilhas, prendendo o algoritmo em ótimos locais. O ajuste dos parâmetros de migração mostrou-se um processo empírico que demanda experimentação cuidadosa.

Falha durante redistribuição parcial. Se o coordenador cair após redistribuir para apenas alguns nós, o novo líder deve definir política clara — reiniciar o ciclo ou assumir o estado parcial — para evitar inconsistências entre as subpopulações dos nós.

Modelagem das mensagens gRPC. Cada rota precisa ser transmitida com fitness, origem, timestamp lógico e número de geração para que o coordenador compare corretamente os indivíduos e aplique o filtro de origem. A definição precisa do contrato no arquivo .proto foi fundamental para a consistência da comunicação em todo o sistema.

Calibração do timeout de eleição. Timeouts curtos geram eleições adulteradas; longos prolongam a indisponibilidade. Esse equilíbrio depende das características da rede e foi tratado como parâmetro configurável, documentado e justificado durante os testes.

4.1 Implementação de tratamento de falhas

Os tratamentos de falha foram implementados em cada fluxo do sistema. Para realizar os testes das respectivas funcionalidades, foi criado um arquivo de simulação, que aborda 4 cenários para validar as falhas tratadas no sistema.

Cenário 1: Coordenador Cai. Este cenário aborda a falta de queda do coordenador inicialmente denominado como M2 durante a execução do sistema. Após a interrupção, os demais nós deixam de receber resposta nas chamadas de monitoramento feitas pelo método ObterStatus e, ao detectar o

timeout, iniciam o Algoritmo do Valentão. Nesse processo, cada nó envia mensagens de eleição apenas para os nós com ID maior que o seu; caso algum nó maior responda, o nó atual entende que existe um candidato com maior prioridade e não assume a liderança. Caso nenhum nó maior responda, o próprio nó se proclama o novo coordenador e notifica os demais por meio da mensagem MensagemLider. Esse cenário permite verificar se, após a falha do coordenador, o nó ativo de maior ID assume corretamente o papel de líder e se o sistema consegue continuar sua execução distribuída sem depender do coordenador inicial.

Cenário 2: Nó Trabalhador Cai. No segundo cenário de falha, foi simulada a queda ou travamento de um nó trabalhador durante a execução do sistema. O objetivo desse teste foi verificar se o mecanismo de threshold do coordenador continua funcionando corretamente mesmo quando um dos nós não responde ou demora excessivamente para enviar seus melhores indivíduos. Como o coordenador trabalha com um threshold de 80%, ele não precisa aguardar obrigatoriamente a resposta de todos os nós para consolidar a migração; ao receber a quantidade mínima esperada de respostas, o processamento pode continuar. Dessa forma, caso um trabalhador fique indisponível, atrase sua execução local ou não consiga se comunicar com o coordenador, o sistema evita ficar bloqueado indefinidamente e mantém a evolução distribuída em andamento. Esse cenário valida a importância do threshold assíncrono para aumentar a robustez do sistema diante de falhas parciais nos nós trabalhadores.

Cenário 3: Dados chegando fora de ordem. No terceiro cenário de falha, foi simulada a chegada de dados fora de ordem durante a comunicação entre os nós e o coordenador. Como o sistema opera de forma assíncrona e utiliza um threshold de 80%, alguns nós podem enviar suas melhores rotas com atraso, fazendo com que informações de ciclos anteriores cheguem depois de dados mais recentes. Para tratar esse problema, cada mensagem de migração carrega um timestamp lógico de Lamport, permitindo que o coordenador compare a ordem dos eventos recebidos. Caso uma mensagem possua timestamp inferior ao último registrado para aquele nó de origem, ela é considerada obsoleta e descartada. Dessa forma, o sistema evita que indivíduos antigos contaminem o pool global de migração e garanta maior consistência na redistribuição das rotas entre as ilhas.

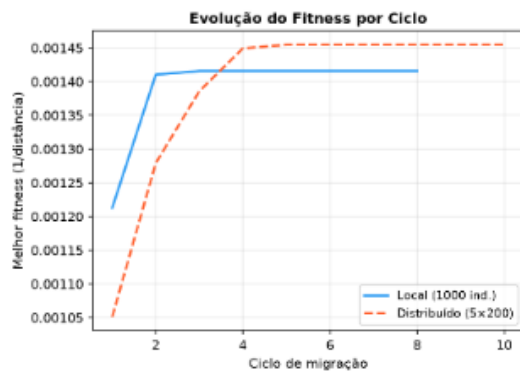
Cenário 4: Coordenador cai em meio a uma redistribuição. No quarto cenário de falha, foi simulada a queda do coordenador durante a etapa de redistribuição dos migrantes para os nós trabalhadores. Nesse caso, o coordenador pode cair depois de receber as melhores populações dos nós, mas antes de enviar os novos migrantes para todos os participantes. Com isso, alguns nós podem ficar

aguardando migrantes que nunca chegam. Para evitar que o sistema fique travado, cada nó utiliza um mecanismo de espera com timeout, implementado com um evento de sincronização, permitindo que a execução continue mesmo sem receber a resposta do coordenador. Ao detectar que os migrantes não chegaram dentro do tempo esperado, o nó encerra a espera, registra o timeout e segue para o próximo ciclo ou participa do processo de eleição de um novo coordenador. Esse cenário valida se o sistema consegue lidar com falhas durante uma redistribuição parcial, evitando bloqueios permanentes e permitindo que a eleição de líder recupere a coordenação da execução distribuída.

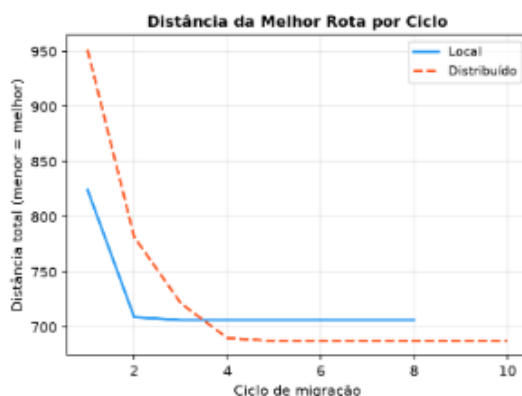
5 Resultados Obtidos

A seguir, será apresentada uma comparação de performance, com a abordagem monolítica, visando acompanhar o resultado de evolução por ciclo de execução do AG, melhores rotas por ciclo e também performance geral. Por fim, é demonstrado um gráfico final, comparando os resultados encontrados anteriormente.

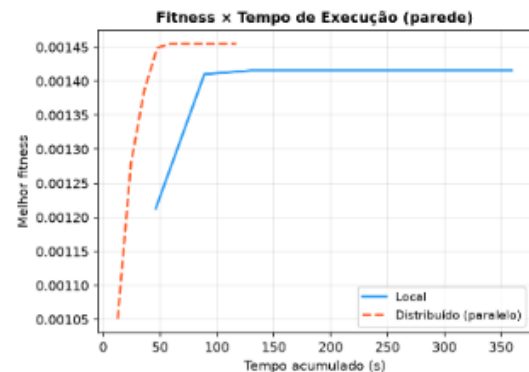
5.1 Evolução por ciclo



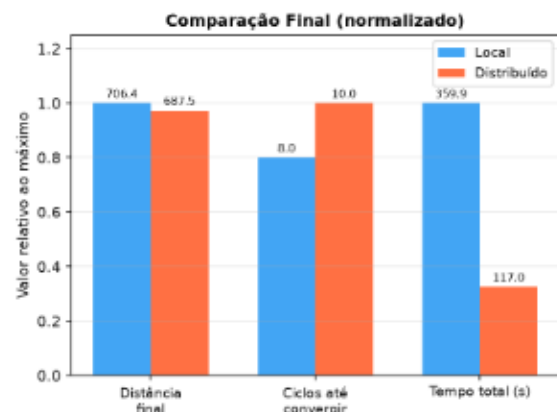
5.2 Distância da Melhor rota encontrada por cada ciclo



5.3 Comparação de tempo de execução e melhor fitness encontrado



5.4 Comparativo final dos resultados anteriores



6 Conclusão

Este trabalho apresentou a implementação de um Algoritmo Genético Distribuído para o Problema do Caixeiro Viajante utilizando o Modelo de Ilhas com comunicação via gRPC, sincronização por Relógio Lógico de Lamport e eleição de coordenador pelo Algoritmo do Valentão. O sistema demonstrou a viabilidade de aplicar conceitos clássicos de computação distribuída — exclusão mútua, detecção de falhas e ordenação causal de eventos — em um problema de otimização combinatória de larga escala.

Os experimentos comparativos confirmaram as vantagens esperadas da abordagem distribuída. A versão distribuída encontrou rotas de menor distância e convergiu em tempo inferior à versão monolítica, com um speed up de cerca de 3.06, resultado que reflete o benefício central do Modelo de Ilhas: ao manter subpopulações evoluindo de forma relativamente independente e trocando material genético periodicamente, o sistema consegue

explorar regiões distintas do espaço de busca em paralelo, reduzindo a probabilidade de convergência prematura para ótimos locais. A versão local, por operar com toda a diversidade concentrada em um único processo sem paralelismo real, tende a homogeneizar a população mais rapidamente.

Vale ressaltar que o desempenho da versão distribuída é sensível à calibração dos parâmetros de migração. A taxa de migração, o tamanho da população por ilha e o intervalo entre ciclos influenciam diretamente o equilíbrio entre exploração e exploração do espaço de soluções, e uma configuração inadequada pode anular os benefícios da distribuição. Neste trabalho, a escolha desses parâmetros foi orientada pela análise do comportamento do sistema durante os experimentos.

Como trabalhos futuros, sugere-se a implementação de um threshold dinâmico no coordenador, que ajuste automaticamente o número mínimo de nós necessários para redistribuição com base nos nós atualmente ativos, conferindo maior resiliência a falhas parciais sem comprometer o aproveitamento dos ciclos de migração. Adicionalmente, propõe-se a investigação de estratégias adaptativas de migração, a extensão do sistema para topologias além da estrela — como anel ou malha — e a validação do desempenho em instâncias com centenas de cidades distribuídas em máquinas fisicamente separadas, cenário no qual os benefícios da paralelização tendem a se manifestar de forma ainda mais expressiva.